CO226-Apr2025 : Database Systems

Lab Tutorial

Indexing and Query Optimization

Indexing

What is an Index?
An index in SQL is similar to an index in a book (list of important topics or words at the end of the book, along with page numbers, so you can quickly find where something is written), it lets you find data without scanning the entire table.

Imagine a table with millions of rows. If you want to find a specific row (imagine a database table with all the students in the Sri Lankan university system and you want to find a student where email = 'e21000@eng.pdn.ac.lk'), SQL would normally have to scan every row. This is called a full table scan. With an index on the email column, SQL can find the row much faster.

Indexes are used to find rows with specific column values quickly. Without an index, MySQL must begin with the first row and then read through the entire table to find the relevant rows. The larger the table, the more this costs. If the table has an index for the columns in question, MySQL can quickly determine the position to seek to in the middle of the data file without having to look at all the data. This is much faster than reading every row sequentially.

How Indexing Works
The database creates a data structure (often a B-tree) that stores the values of the indexed column in a sorted manner. Along with the values, the index keeps pointers to the actual rows in the table.

> What is a B-tree
>
> > A tree data structure that is popular for use in database indexes. The structure is kept sorted at all times, enabling fast lookup for exact matches (equals operator) and ranges (for example, greater than, less than, and BETWEEN operators). This type of index is available for most storage engines, such as InnoDB and MyISAM.
> > Because B-tree nodes can have many children, a B-tree is not the same as a binary tree, which is limited to 2 children per node.

When to Use Indexes
Indexes improve performance in

- SELECT queries with WHERE conditions
- JOIN operations
- ORDER BY or GROUP BY clauses
- Searching in large tables

Issues with indexing

- Slower INSERT, UPDATE, DELETE. Because the index also needs to be updated.
- Takes extra disk space
- Too many indexes can reduce performance rather than help.

When to Enable Indexing?
Indexing is only advantageous for huge tables with regularly accessed information. Without indexes, when tables grow to enormous proportions, response times suffer from queries targeting those obtuse tables. Inefficient queries manifest into latency within application or website performance.

We commonly identify this latency by using the MySQL slow query log feature. Once a colossal table hits its tipping point, it reaches the potential for downtime for applications and websites. Conducting routine evaluations for growing databases establishes optimal database performance and sidesteps long queries' inherent interruptions.

What Information Does One Index?
Selecting what to index is probably the most challenging part to indexing your databases. Determining what is important enough to index and what is benign enough to not index. Generally speaking, indexing works best on those columns that are the subject of the WHERE clauses in your commonly executed queries.

Consider a table with the below columns

- ID
- TITLE
- LAST_NAME
- FIRST_NAME
- DOB
- GENDER
- AGE

If your queries rely on testing the WHERE clause using LAST_NAME and FIRST_NAME then indexing by these two columns would significantly increase query

response times. Alternatively, if your queries rely on a simple ID lookup, indexing by ID would be the better choice.

Use of Indexes in MySQL

- To find the rows matching a WHERE clause quickly.
- To eliminate rows from consideration. If there is a choice between multiple indexes, MySQL normally uses the index that finds the smallest number of rows (the most selective index).
- If the table has a multiple-column index, any leftmost prefix of the index can be used by the optimizer to look up rows. For example, if you have a three-column index on (col1, col2, col3), you have indexed search capabilities on (col1), (col1, col2), and (col1, col2, col3). For more information, click, Multiple-Column Indexes.
- To retrieve rows from other tables when performing joins. MySQL can use indexes on columns more efficiently if they are declared as the same type and size. In this context, VARCHAR and CHAR are considered the same if they are declared as the same size. For example, VARCHAR(10) and CHAR(10) are the same size, but VARCHAR(10) and CHAR(15) are not.
- more

<u>Managing Indexes</u>

Now lets' see some of the basics of manipulating indexes using MySQL syntax. In examples, we will include the creation, deletion, and listing of indexes.

- **Listing/Showing Indexes**

  Tables can have multiple indexes. The syntax for viewing an index is below.

  SHOW INDEX FROM *tableName*;

  SHOW INDEX FROM users;

- **Creating an Index**

  Index creation has a simple syntax. The difficulty is in determining what columns need Indexing

  As previously mentioned, tables can have multiple indexes. Multiple indexing is useful for creating indexes attuned to the queries required by your application or website. Indexes can be created during a table's creation or added onto the table as additional indexes later on. We will go over both methods below.

  CREATE TABLE *tableName* (
  …
  …
  …
  INDEX *indexName* (*columnName*)
  );

  CREATE TABLE users (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
  INDEX index_email (email)
  );

  CREATE INDEX *indexName* ON *tableName*(*columnName*);

  CREATE INDEX index_email ON users(email);

- **Deleting Indexes**
  While managing indexes, you may find it necessary to remove some. Deleting indexes is also a very simple process

  DROP INDEX [index_name] ON [table_name];

  DROP INDEX index_email ON users(email);

There are different types of indexes

- Composite indexes
  - A composite index is created on two or more columns of a table. It helps speed up queries that filter or sort by multiple columns together.
  - You often query with multiple conditions in the WHERE, JOIN, or ORDER BY clause using more than one column.
  - The order of columns matters.
    - CREATE INDEX index_col1_col2 ON tableName(col1, col2);
    - col1 alone √
    - col1 and col2 together √
    - col2 alone ╳ (not helpful unless a separate index exists)
    - click here to see an example
- Unique indexes
  - A unique index ensures that all the values in the indexed columns are distinct, no duplicates allowed.
  - To enforce data integrity (email, username, SSN)
  - **Automatically created when you define a column as UNIQUE or PRIMARY KEY.**
  - Trying to insert a duplicate value will cause an error.
  - Good for fields where duplicates don't make sense.
    - For example,
      A student table where E number is the primary key and email is also given, we can create a UNIQUE INDEX for the email
      Note that creating an index and creating a unique index are two different things
- Full-text indexes (for searching text)
  - A full-text index allows for advanced searching of text-based data, such as finding words or phrases inside large text fields.
  - When you want to support natural language search, like searching in articles, blogs, descriptions, etc.

Indexes are less important for queries on small tables, or big tables where report queries process most or all of the rows. When a query needs to access most of the rows, reading sequentially is faster than working through an index. Sequential reads minimize disk seeks, even if not all the rows are needed for the query.

References

- https://dev.mysql.com/doc/refman/8.0/en/mysql-indexes.html
- https://www.w3schools.com/sql/sql_create_index.asp

Query Optimization

The EXPLAIN command shows you the query execution plan It is a breakdown of how MySQL plans to retrieve the requested data.

It tells you how the query will run step by step

- Which tables are read first
- What type of join is used
- Whether indexes are being used
- How many rows are being examined

When a SELECT query does not perform well as you think it should, use the EXPLAIN statement to ask the MySQL server for information about how the query optimizer processes the query.

When you issue a query, the MySQL Query Optimizer tries to devise an optimal plan for query execution. EXPLAIN is one of the most powerful tools available for understanding and optimizing inefficient queries.

You can use EXPLAIN by placing the keyword EXPLAIN in front of SELECT, DELETE, INSERT, REPLACE, and UPDATE query as of MySQL5.6.3. Earlier versions permit only SELECT statements for this. You can also add the keyword EXTENDED after EXPLAIN in your query for additional information about the execution plan.

Basic Syntax

EXPLAIN <rest of the SQL query>

Now let's elaborate the output of EXPLAIN using an example. **You can also try using EXPLAIN on one of your queries on one of your tables so it would be easier for you to understand. First run the query normally and then run it with EXPLAIN in front.**

```
mysql>
mysql>
mysql> select * from students;
+------------+--------+------------+----------------------+------+
| student_id | name   | dob        | email                | gpa  |
+------------+--------+------------+----------------------+------+
|         49 | Lahiru | 2002-09-01 | e21049@eng.pdn.ac.lk | NULL |
|        138 | FIKRY  | 2002-01-15 | e21138@eng.pdn.ac.lk | 3.95 |
|        217 | Ishan  | 2002-06-21 | e21217@eng.pdn.ac.lk | 3.35 |
+------------+--------+------------+----------------------+------+
3 rows in set (0.04 sec)

mysql> select * from students where student_id = 49;
+------------+--------+------------+----------------------+------+
| student_id | name   | dob        | email                | gpa  |
+------------+--------+------------+----------------------+------+
|         49 | Lahiru | 2002-09-01 | e21049@eng.pdn.ac.lk | NULL |
+------------+--------+------------+----------------------+------+
1 row in set (0.00 sec)

mysql> EXPLAIN select * from students where student_id = 49;
+----+-------------+----------+------------+-------+---------------+---------+---------+-------+------+----------+-------+
| id | select_type | table    | partitions | type  | possible_keys | key     | key_len | ref   | rows | filtered | Extra |
+----+-------------+----------+------------+-------+---------------+---------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | students | NULL       | const | PRIMARY       | PRIMARY | 4       | const |    1 |   100.00 | NULL  |
+----+-------------+----------+------------+-------+---------------+---------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.00 sec)

mysql>
mysql>
mysql>
mysql>
```

🔗Key Columns in EXPLAIN Output

Column	JSON Name	Meaning
`id`	`select_id`	The `SELECT` identifier
`select_type`	None	The `SELECT` type
`table`	`table_name`	The table for the output row
`partitions`	`partitions`	The matching partitions
`type`	`access_type`	The join type
`possible_keys`	`possible_keys`	The possible indexes to choose
`key`	`key`	The index actually chosen
`key_len`	`key_length`	The length of the chosen key
`ref`	`ref`	The columns compared to the index
`rows`	`rows`	Estimate of rows to be examined
`filtered`	`filtered`	Percentage of rows filtered by table condition
`Extra`	None	Additional information

Since that sounds a bit too formal, here is a somewhat easy to understand meanings of important columns,

Column	Meaning
id	Step number in the query (1 = first step)
select_type	Type of SELECT (SIMPLE, SUBQUERY)
table	The table being read
type	shows efficiency (**ALL, ref, index, const**)
possible_keys	Indexes MySQL could use
key	Index MySQL actually used
rows	Estimated number of rows to examine
Extra	

🔗**type is especially important**

It tells how efficiently MySQL is reading data

type	Meaning	Performance
ALL	Full table scan	Very slow
index	Full index scan	Slower
range	Index range scan	Good
ref	Index lookup for matching rows	Better
const	Single-row lookup by PK/unique	Best

How EXPLAIN Helps in Query Optimization

- Diagnose full table scans
- Check if indexes are used
- Understand joins and subquery behavior
- Estimate row scans and query cost
- Spot inefficient patterns like filesort or temporary tables

References

- https://www.eversql.com/mysql-explain-example-explaining-mysql-explain-using-stackoverflow-data/
- https://dev.mysql.com/doc/refman/5.7/en/using-explain.html
- https://dzone.com/articles/how-to-optimize-mysql-queries-for-speed-and-perfor